

CT1 Exercises Data Transfer

1. What is a load/store architecture?

2. What is the difference between a mov and a movs instruction?

3. Which data transfer instructions should you use if at least one high register is an operand?

4. List different ways of initializing a low register with an immediate value. What are the advantages/disadvantages?

5. What is a pseudo-Instruction? Explain what is done with a <ldr Rn, =literal> pseudo-instruction.

6. Write down the assembly instructions to perform the following actions.

7. Copy contents of r1 in r3 (flags unchanged)

8. Initialize r0 with 0xAA (flags unchanged)

9. Initialize r1 with 234 (flags modified)

10. Initialize r4 with 0x55AACC

11. Copy contents of r9 in r3.

12. Initialize r10 with 0x345678

1. Copy contents of r8 in r9

13. Whenever possible, work out the contents of registers or memory positions that have changed.

again:	ldr	r0,=0xFF	
	ldr	r1,lit1	
	ldr	r2,lit2	
	ldr	r3,=0x55AA	
	mov	r4,r1	
	mov	r4,r2	
	mov	r6,r3	
	movs	r7, #04	
	ldr	r0,=lit1	r0 = address of lit1 (Address is not known)
	ldr	r1,=lit2	
	ldr	r2,=lit3	
	ldrb	r5,[r2]	
	ldrh	r6,[r2,#2]	
	ldr	r2,[r0]	
	ldr	r3,[r0,#4]	
	str	r3,[r1]	
	str	r2,[r1,#8]	
	str	r4,[r1,r7]	
	ldr	r5,=lit5	
	movs	r0,#0	
	adds	r7,r0,#1	
	ldrsb	r6,[r5,r0]	
	ldrsh	r6,[r5,r7]	
	b	Again	
lit1:	.word	0xEFAA	
lit2:	.word	0x12345	
lit3:	.word	0x8F1097	
lit4:	.word	0xFF76552F	
lit5:	.word	0xAA654389	
lit6:	.word	0x0165	
Var1:	.word	0x23	
Var2:	.word	0x24	
Var3:	.word	0x23455678	
Var4:	.word	0xE4568900	

14. Consider the following assembled listing. Use table below for results of a) and b).

- For all the `ldr` instructions, calculate the value of `imm` and write the instructions in the form `ldr <Rt>, [pc, #<imm>]` (give `imm` in hex or decimal, as you want)
- Replace `yyyy` with the appropriate machine code
- Make sure you understand what the `<b loop>` instruction does.
- What do you think will happen during the program execution if there was no `<b loop>` instruction after `add r0, r1`? (Check the slides if you do not know.)

```

                                .equ CONST_VALUE_X, 123456
00000000
00000000          loop:
00000000      yyyy          ldr    r0, =0xFF55AAB0
00000002      yyyy          ldr    r1, =CONST_VALUE_X
00000004      yyyy          ldr    r2, myval2
00000006      yyyy          ldr    r3, myval3
00000008      yyyy          ldr    r4, myval4
0000000A      yyyy          ldr    r5, myval5
0000000C      yyyy          ldr    r6, myval6
0000000E      yyyy          ldr    r7, myval7
00000010      4408          add    r0, r1
00000012      E7F5          b      loop
00000014      33445566      myval3: .word    0x33445566
00000018      0000FFAA      myval4: .word    0x00FFAA
0000001C      00110044      myval5: .word    0x110044
00000020      00AABBCC      myval7: .word    0xAABBCC
                                @ Beware the order of variables
00000024      7788ABCD      myval6: .word    0x7788ABCD
00000028      33445555      myval2: .word    0x33445555
0000002C      FF55AAB0
                                00123456

```

`imm` is shown in decimal and hex to allow you to see both.

Code (yyyy)	imm in hexadecimal	imm in decimal
480A	<code>ldr r0, [pc, #0x28]</code>	<code>ldr r0, [pc, #40]</code>
Example		

Note: Remember that the `pc` contains the address of the next instruction. If the address is not word-aligned, the next upper word-address will be used for calculation.

`<b loop>` branches unconditionally to `loop`. Without this, the CPU will continue to fetch and execute and take the following contents of memory as code and try to interpret and execute it. Dangerous!

15. In the following C program, code the statements in **bold green** in assembly.

```
// C-Code
uint32_t x;
uint32_t y;
uint32_t *xp;

void main(void) {
    x = 3;
    xp = &x;
    y = *xp;
}
```



```
AREA myVar, DATA, READWRITE

@ Definition of variables in RAM space.
X      .word      0
Y      .word      0
XP     .word      0

AREA myCode, CODE, READONLY

main:
```

16. In the following C program, code the statements in **bold green** in assembly. Use the frame for solution on following page.

```
// C-Code
uint8_t demoArray[2];
uint8_t *xp;

void main(void) {
    demoArray[0] = 10;

    demoArray[1] = 11;

    xp = demoArray;

    *xp = 111;

    xp++;

    *xp = 112;
}
```

Note: Other statements of the C program are shown in frame with machine code and addresses as examples to help you.

You are only required to write the assembly code (not the machine code).

Frame for solution

```
22:      demoArray[0] = 10;
0x08000254 200A      movs      r0,#0x0A
0x08000256 4909      ldr       r1,[pc,#36] ; @0x0800027C
0x08000258 7008      strb      r0,[r1,#0x00]

23:      demoArray[1] = 11;

24:      xp = demoArray;
0x0800025E 4608      mov       r0,r1
0x08000260 4907      ldr       r1,[pc,#28] ; @0x08000280
0x08000262 6008      str       r0,[r1,#0x00]
25:      *xp = 111;
0x08000264 206F      movs      r0,#0x6F
0x08000266 6809      ldr       r1,[r1,#0x00]
0x08000268 7008      strb      r0,[r1,#0x00]

26:      xp++;

27:      *xp = 112;

28: }
0x0800027A 4770      bx         lr ;Not part of solution
0x0800027C 0000      .hword     0x0000
0x0800027E 2000      .hword     0x2000
0x08000280 0004      .hword     0x0004
0x08000282 2000      .hword     0x2000
```

17. Consider the following listing.

```

08000008 4804      again  ldr    r0, lit1
0800000A 480A      ldr    r0, =lit1
0800000C 4A04      ldr    r2, lit2
0800000E 4A0A      ldr    r2, =lit2
08000010 4B04      ldr    r3, lit3
08000012 4C04      ldr    r4, lit3
08000014 4E09      ldr    r6, =lit4
08000016 4F09      ldr    r7, =lit4
08000018 4408      add    r0, r1
0800001A E7F5      b      again

0800001C 00000001  xxx    .word    0x01 @ xxx stands for a label
08000020 00000002  xxx    .word    0x02
08000024 00000003  xxx    .word    0x03
08000028 00000004  xxx    .word    0x04
0800002C 00000005  xxx    .word    0x05
08000030 00000006  xxx    .word    0x06
08000034
          00000000
          00000000
          00000000
          00000000

```

- a) Compute on the basis of the opcode (marked in blue) the position where data is read to load registers (see table with example)

Instruction address	Opcode (marked in blue)	position where data is read	Contents of affected register after operation
08000008	4804	0x0800001C	r0 = 0x00000001

- b) The opcodes for <ldr r0, lit1> and <ldr r0, =lit1> are the same. But where are the operands? Explain the differences between the 2.
- c) Compute the contents of the registers after each instruction is executed once. Write results in table in a).